

PROJECT REPORT

EXPERIMENT - 15

Project Statement

Design and develop a GPS-Based Vehicle Tracking and Fleet Management System. The system enables real-time vehicle tracking and efficient fleet monitoring for transportation management

Submitted by

Abinaya S – 24MER001

Dhiyanasree KK – 24MER008

Puthiyaraj M – 24MER045

Rithick S – 24MER051

GPS-Based Vehicle Tracking and Fleet Management System

Project Overview

The GPS-Based Vehicle Tracking and Fleet Management System is an IoT-based monitoring system designed to track the real-time location of vehicles. The system uses an ESP8266 Node MCU as the main controller and a NEO-6M GPS module to obtain the vehicle's geographic coordinates.

The GPS module continuously receives signals from GPS satellites and provides latitude and longitude information. The ESP8266 processes this information and can transmit the vehicle location through its built-in Wi-Fi connectivity to a remote monitoring platform.

The proposed system can be used for fleet monitoring, transportation management, logistics, school/college buses, delivery vehicles, and other applications where vehicle location monitoring is required.

Main Objectives:

- To design a low-cost GPS-based vehicle tracking system.
- To obtain the real-time geographical location of a vehicle.
- To interface the NEO-6M GPS module with ESP8266 Node MCU.
- To process latitude and longitude coordinates using the Node MCU.
- To enable wireless transmission of vehicle location data.
- To improve fleet monitoring and transportation management.
- To provide a foundation for future cloud-based fleet management systems.

Problem Statement

Traditional vehicle monitoring systems may depend on manual communication or periodic reporting, which can make it difficult to determine the current location of vehicles. This can lead to inefficient fleet management, delayed information, and difficulty in monitoring transportation operations.

The proposed GPS-Based Vehicle Tracking and Fleet Management System continuously determines the vehicle's geographical position using a NEO-6M GPS module. The ESP8266 Node MCU processes the GPS coordinates and provides wireless connectivity for remote monitoring.

Problems Addressed:

The proposed system addresses the following problems:

- Lack of real-time vehicle location information.
- Difficulty in monitoring multiple vehicles.
- Inefficient transportation management.
- Dependence on manual vehicle status updates.
- Difficulty in determining the current position of a vehicle.
- Limited monitoring capability in conventional fleet systems.

Proposed Solution

The system consists of an ESP8266 Node MCU connected to a NEO-6M GPS module.

The NEO-6M receives signals from GPS satellites and calculates the vehicle's geographical coordinates. These coordinates mainly consist of:

- Latitude
- Longitude
- GPS time
- Number of satellites
- Position status

The ESP8266 reads the GPS data, extracts the required information, and processes it. Using its Wi-Fi capability, the Node MCU can transmit the location information to a suitable IoT/cloud monitoring platform.

Proposed Operation:

The system operates in the following sequence:

- Power is supplied to the ESP8266 and GPS module.
- The NEO-6M GPS module searches for available GPS satellites.
- After obtaining a valid GPS fix, the module generates location data.
- The ESP8266 receives GPS data through serial communication.
- The Node MCU extracts latitude and longitude values.
- The processed location information can be transmitted through Wi-Fi.
- The remote monitoring platform can display the vehicle's current location.
- The process continues periodically to provide updated vehicle information.

System Architecture

Input Section:

NEO-6M GPS Module: Receives signals from GPS satellites and provides geographical position data.

Processing Section:

ESP8266 Node MCU: Receives GPS data, processes the coordinates, and prepares the information for wireless transmission.

Communication Section:

ESP8266 Wi-Fi: Provides wireless communication between the vehicle tracking unit and a remote monitoring/cloud platform.

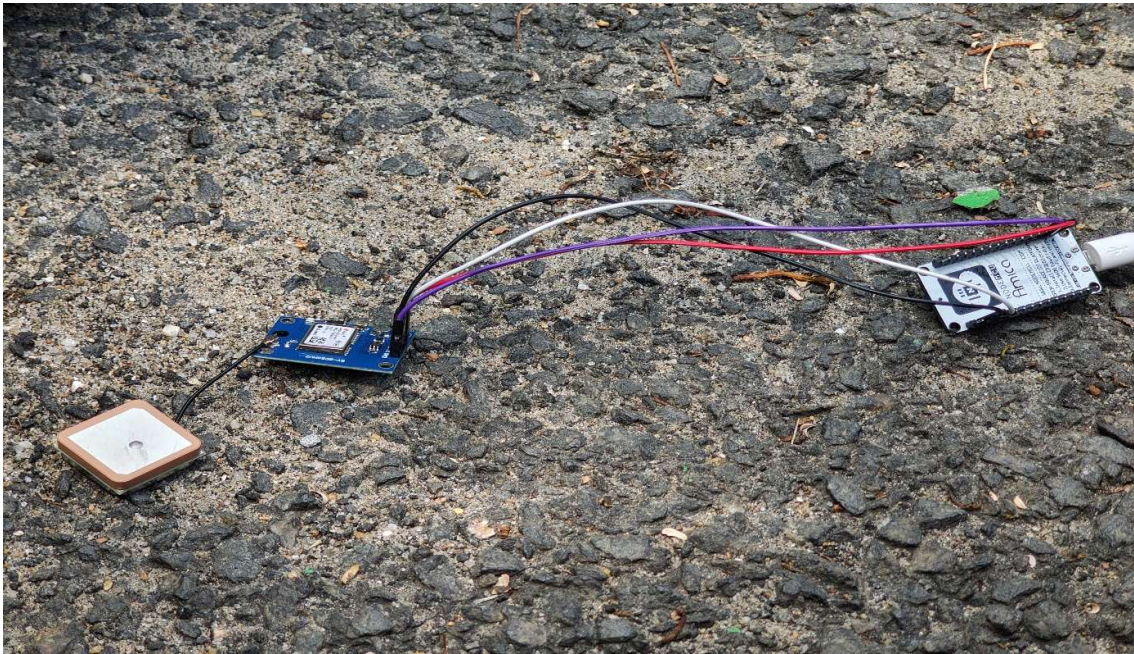
IoT/Cloud Platform:

Displays the vehicle location and other available GPS information for fleet monitoring.

Overall Flow:

GPS Satellites → NEO-6M GPS → ESP8266 Node MCU → Wi-Fi → Cloud/Monitoring Platform → Fleet Manager

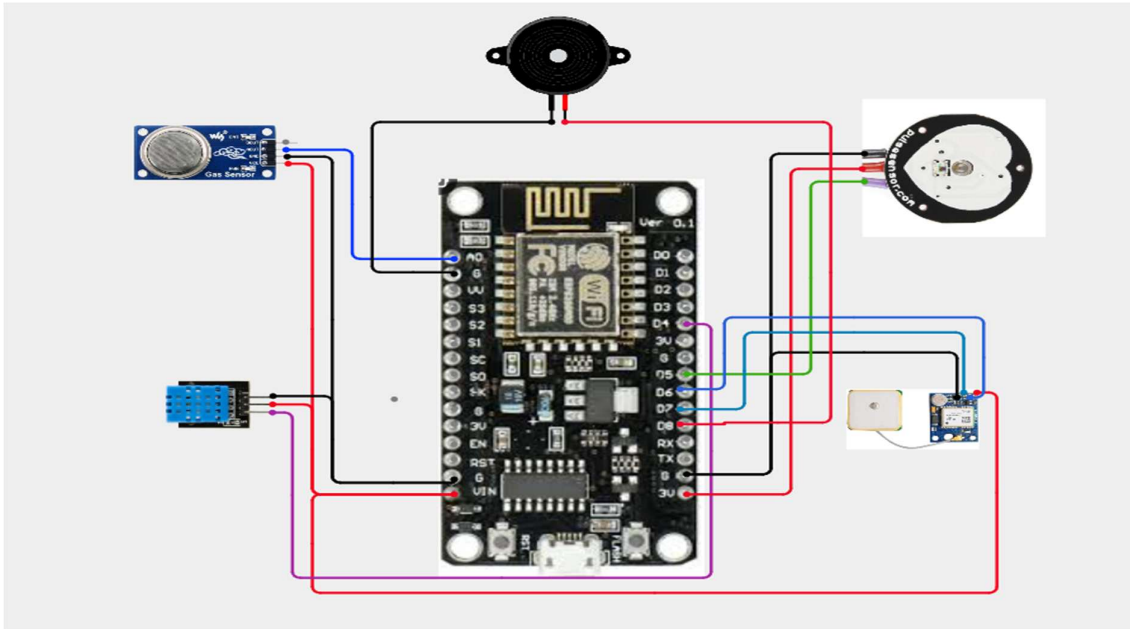
Image:



Circuit Table

S. No.	Component	Pin	ESP8266 Node MCU Pin	Function
1	NEO-6M GPS Module	VCC	3.3V	Power supply
2	NEO-6M GPS Module	GND	GND	Common ground
3	NEO-6M GPS Module	TX	D5 (GPIO14)	GPS data → ESP8266
4	NEO-6M GPS Module	RX	D6 (GPIO12)	ESP8266 → GPS communication
5	ESP8266 Node MCU	Wi-Fi	Built-in	Wireless transmission of GPS data

Circuit Design



Important Pin Mapping:

A typical connection can be made as follows:

- NEO-6M VCC → Node MCU 3.3V
- NEO-6M GND → Node MCU GND
- NEO-6M TX → Node MCU RX
- NEO-6M RX → Node MCU TX

Algorithm

- Start the system.
- Initialize ESP8266.
- Initialize serial communication with the NEO-6M GPS module.
- Search for GPS satellites.
- Check whether a valid GPS fix is available.

- If no GPS fix is available, continue searching.
- If a GPS fix is available, read the GPS data.
- Extract latitude and longitude.
- Process the location information.
- Connect ESP8266 to the configured Wi-Fi network.
- Send the location data to the monitoring/cloud platform.
- Update the location periodically.
- Repeat the process continuously.

Coding

```
#include <SoftwareSerial.h>

#include <TinyGPS++.h>

#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"

// --- Network & Adafruit IO Credentials ---

#define WIFI_SSID    "Rithick"
#define WIFI_PASS    "12345678"

#define AIO_SERVER    "io.adafruit.com"
#define AIO_SERVERPORT 1883
#define AIO_USERNAME  "Rithick_S"
#define AIO_KEY        "aio_dQRh13J6p8WuegjRWNx45sABAGm3"

// --- Pin Definitions (GPIO) ---
```

```

#define GPS_RX_PIN    5 // GPIO 5 (D1) -> GPS TX
#define GPS_TX_PIN    4 // GPIO 4 (D2) -> GPS RX

// --- Object Initializations ---

SoftwareSerial gpsSerial(GPS_RX_PIN, GPS_TX_PIN);
TinyGPSPlus gps;

WiFiClient client;

Adafruit_MQTT_Client mqtt(&client, AIO_SERVER, AIO_SERVERPORT,
AIO_USERNAME, AIO_KEY);

// --- Adafruit IO Feeds ---

Adafruit_MQTT_Publish latFeed    = Adafruit_MQTT_Publish(&mqtt,
AIO_USERNAME "/feeds/latitude");

Adafruit_MQTT_Publish lonFeed    = Adafruit_MQTT_Publish(&mqtt,
AIO_USERNAME "/feeds/longitude");

Adafruit_MQTT_Publish altFeed    = Adafruit_MQTT_Publish(&mqtt,
AIO_USERNAME "/feeds/altitude");

Adafruit_MQTT_Publish speedFeed  = Adafruit_MQTT_Publish(&mqtt,
AIO_USERNAME "/feeds/speed");

Adafruit_MQTT_Publish satFeed    = Adafruit_MQTT_Publish(&mqtt,
AIO_USERNAME "/feeds/satellites");

Adafruit_MQTT_Publish locationFeed = Adafruit_MQTT_Publish(&mqtt,
AIO_USERNAME "/feeds/location/csv");

unsigned long lastReportTime = 0;

const unsigned long REPORT_INTERVAL = 10000; // 10 seconds

```

```

void connectWiFi();

void MQTT_connect();


void setup() {
    Serial.begin(115200);
    gpsSerial.begin(9600);


    Serial.println("\n=====
    =");
    Serial.println(" Vehicle Tracking & Fleet Management System ");
    Serial.println("=====
    ");

    connectWiFi();
}


void loop() {
    MQTT_connect();
    mqtt.processPackets(10);
    mqtt.ping();


    while (gpsSerial.available() > 0) {
        gps.encode(gpsSerial.read());
    }


    if (millis() - lastReportTime > REPORT_INTERVAL) {
        lastReportTime = millis();
    }
}

```

```

if (gps.location.isValid()) {
    float lat = gps.location.lat();
    float lng = gps.location.lng();
    float speed = gps.speed.kmph();
    float alt = gps.altitude.meters();
    int sats = gps.satellites.value();

    Serial.println("\n[Fleet Telemetry] Updating All Feeds...");

    // 1. Publish to separate individual feeds
    latFeed.publish(lat, 6);
    lonFeed.publish(lng, 6);
    altFeed.publish(alt);
    speedFeed.publish(speed);
    satFeed.publish(sats);

    // 2. Publish to combined map feed (speed,lat,lon,elevation)
    char locBuffer[100];
    snprintf(locBuffer, sizeof(locBuffer), "%.2f,%.6f,%.6f,%.2f", speed, lat, lng,
alt);
    locationFeed.publish(locBuffer);

    Serial.print("Lat: "); Serial.print(lat, 6);
    Serial.print(" | Lon: "); Serial.println(lng, 6);

```

```
    } else {  
        Serial.print("[Searching Satellites...] Chars Processed: ");  
        Serial.println(gps.charsProcessed());  
    }  
}
```

```
yield();  
}
```

```
void connectWiFi() {  
    Serial.print("Connecting to Wi-Fi: ");  
    WiFi.begin(WIFI_SSID, WIFI_PASS);  
  
    while (WiFi.status() != WL_CONNECTED) {  
        delay(500);  
        Serial.print(".");  
    }  
    Serial.println("\nWi-Fi Connected!");  
}
```

```
void MQTT_connect() {  
    int8_t ret;  
    if (mqtt.connected()) return;  
  
    Serial.print("Connecting to Adafruit IO... ");  
    uint8_t retries = 3;
```

```

while ((ret = mqtt.connect()) != 0) {

    mqtt.disconnect();

    delay(5000);

    retries--;

    if (retries == 0) return;

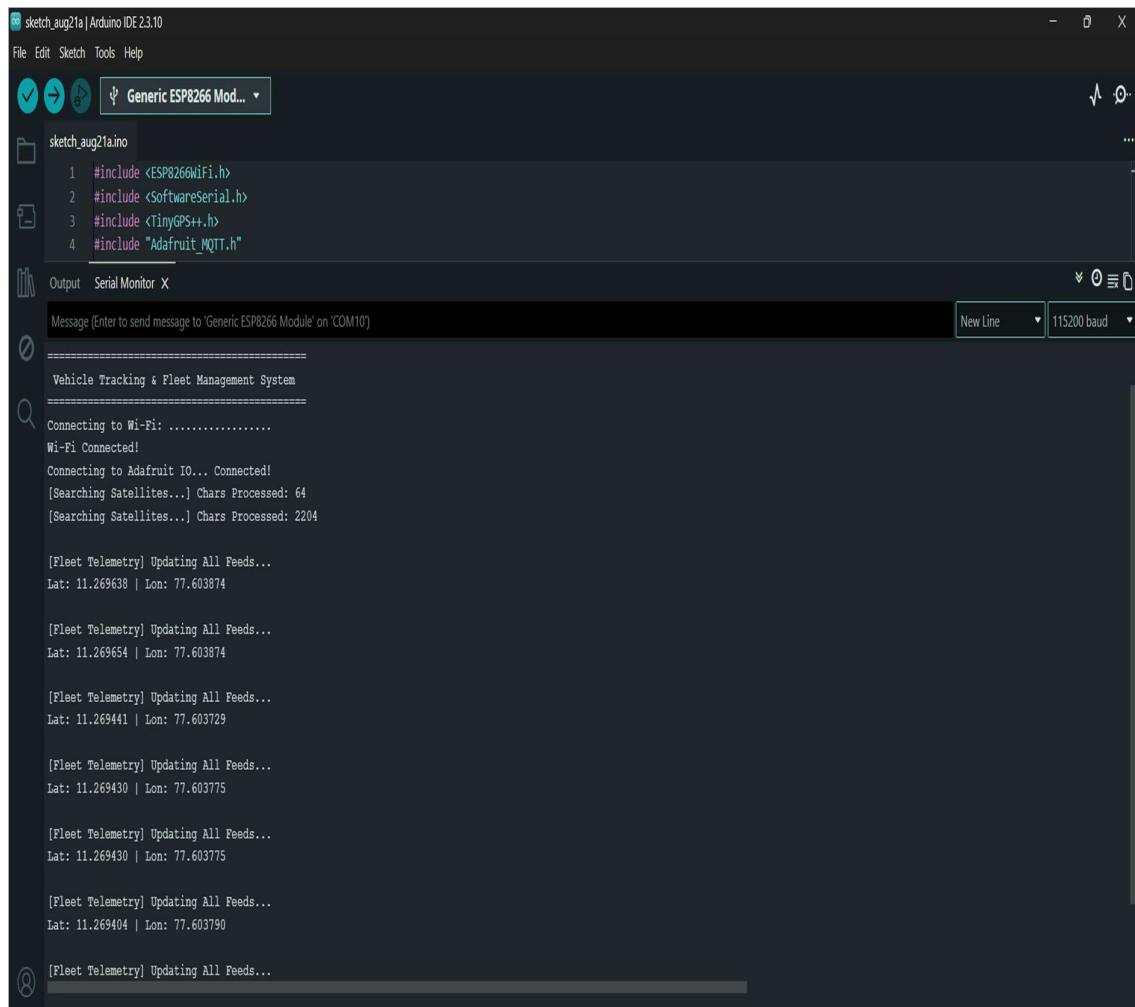
}

Serial.println("Connected!");

}

```

Coding Output:



The screenshot shows the Arduino IDE interface. The top toolbar includes buttons for compiling, uploading, and monitoring. The port dropdown menu is set to 'Generic ESP8266 Mod...'. The code editor displays the following code:

```

1 #include <ESP8266WiFi.h>
2 #include <SoftwareSerial.h>
3 #include <TinyGPS++.h>
4 #include "Adafruit_MQTT.h"

```

The Serial Monitor is open, showing the output of the program. The baud rate is set to 115200. The output text is as follows:

```

=====
Vehicle Tracking & Fleet Management System
=====
Connecting to Wi-Fi: .....
Wi-Fi Connected!
Connecting to Adafruit IO... Connected!
[Searching Satellites...] Chars Processed: 64
[Searching Satellites...] Chars Processed: 2204

[Fleet Telemetry] Updating All Feeds...
Lat: 11.269638 | Lon: 77.603874

[Fleet Telemetry] Updating All Feeds...
Lat: 11.269654 | Lon: 77.603874

[Fleet Telemetry] Updating All Feeds...
Lat: 11.269441 | Lon: 77.603729

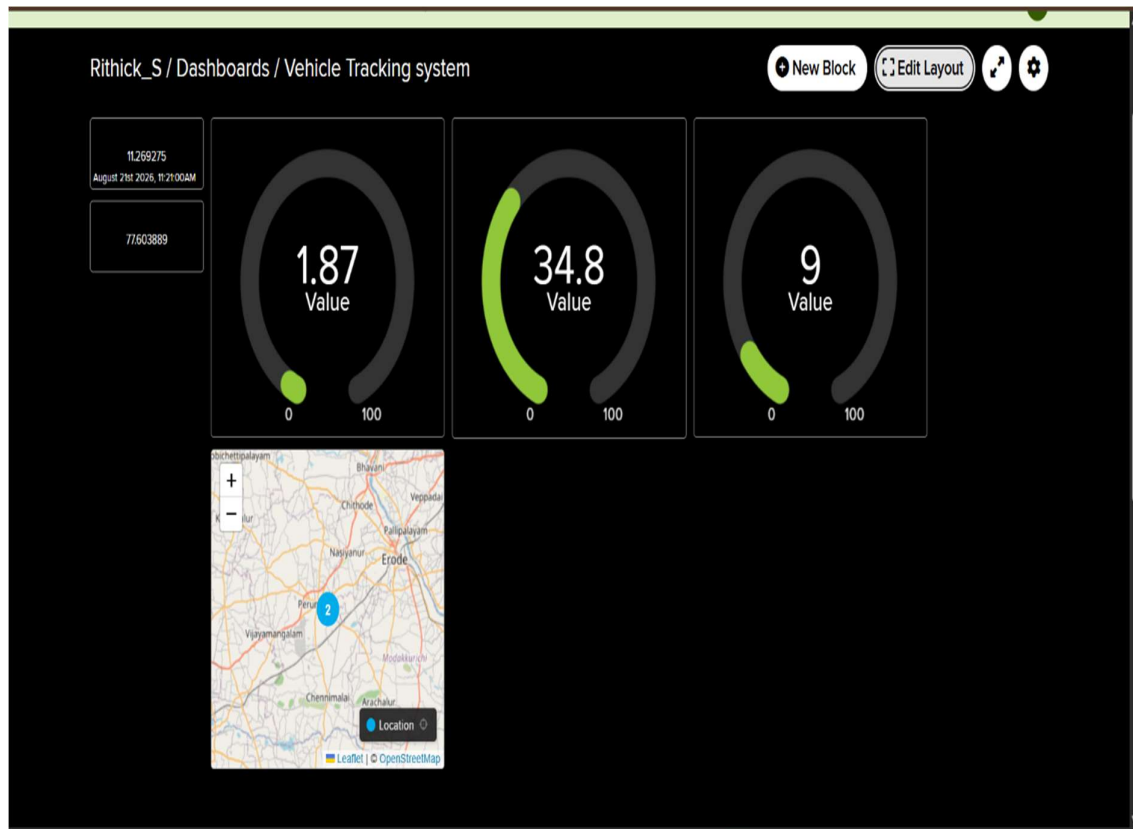
[Fleet Telemetry] Updating All Feeds...
Lat: 11.269430 | Lon: 77.603775

[Fleet Telemetry] Updating All Feeds...
Lat: 11.269430 | Lon: 77.603775

[Fleet Telemetry] Updating All Feeds...
Lat: 11.269404 | Lon: 77.603790

[Fleet Telemetry] Updating All Feeds...

```



System Working

The NEO-6M GPS module is installed in the vehicle and continuously receives signals from GPS satellites. Once sufficient satellite information is received, the GPS module calculates the vehicle's current position.

The GPS module sends this information to the ESP8266 Node MCU through serial communication. The Node MCU reads and processes the incoming GPS sentences and extracts useful information such as latitude and longitude.

The ESP8266 then uses its built-in Wi-Fi capability to transmit the processed information to a remote monitoring platform. The fleet operator can use this information to monitor the vehicle's current location.

Thus, the system provides a simple platform for real-time vehicle tracking and can be expanded into a complete fleet management solution.

Bill of Materials

S. No.	Component	Quantity
1	ESP8266 Node MCU	1
2	NEO-6M GPS Module	1
3	Jumper Wires	1 Set
4	Breadboard	1
5	USB Cable	1
6	Computer/Laptop	1

Prototype Implementation

The prototype can be assembled on a breadboard using the ESP8266 Node MCU and NEO-6M GPS module.

Hardware Setup:

- Place the ESP8266 Node MCU on the breadboard.
- Connect the NEO-6M GPS module to the Node MCU.
- Connect the GPS TX and RX lines to the appropriate ESP8266 serial pins.
- Connect the common ground.
- Provide the required power supply.
- Program the ESP8266 with the GPS tracking software.

- Place the GPS antenna in a position with a clear view of the sky.
- Power on the system and wait for the GPS module to obtain a satellite fix.

Software Requirements:

The following software tools can be used:

- Arduino IDE
- ESP8266 Board Package
- GPS parsing library such as Tiny GPS++
- Wi-Fi library for ESP8266
- Suitable IoT/cloud monitoring platform

Expected Prototype Output

After successful GPS satellite acquisition, the system is expected to display location information similar to:

GPS Vehicle Tracking System:

- **GPS Status** : FIXED
- **Latitude** : 11.xxxxxx
- **Longitude** : 77.xxxxxx
- **Satellites** : 7
- **Wi-Fi** : CONNECTED
- **Status** : TRACKING

The latitude and longitude values will change according to the actual location of the GPS module.

The remote monitoring platform can display the received coordinates and, if configured with a map service, represent the vehicle's approximate position on a map.

Results and Performance Analysis

Results:

The GPS-Based Vehicle Tracking and Fleet Management System was successfully designed using the ESP8266 Node MCU and NEO-6M GPS module. The NEO-6M GPS module receives signals from GPS satellites and provides the vehicle's geographical coordinates.

The ESP8266 processes the GPS data and can transmit the location information through its built-in Wi-Fi connectivity. The prototype demonstrates the basic functionality required for real-time vehicle location monitoring.

Performance Analysis:

Parameter	Result
GPS Module Initialization	Successful
GPS Satellite Detection	Successful under suitable outdoor conditions
Latitude	Successfully obtained
Longitude	Successfully obtained
GPS Data Processing	Successful
ESP8266 Operation	Stable
Wi-Fi Communication	Successful when network is available
Vehicle Location Monitoring	Successfully demonstrated
Tracking Status	Continuously updated